INTRODUCTION TO SOFTWARE ENGINEERING (COM 324) HND I COMPUTER SCIENCE,

1. Fundamental of Software Engineering
2. The need for Software Engineering
3. Software Evolution
4. Software Process and Models
5. Software Requirements
6. Software Design Process
7. Software Development
8. Software Testing
9. Software Management
10. Characteristics of good software

INTRODUCTION

Software engineering is a term formed from two words: **software** and **engineering**. Software engineering as a single term will require defining the two terms separately. **Software** is defined as a collection of codes written in a specific programming language to perform a specific task and meet a specific requirement. **Engineering** is the process of creating products using the most up-to-date techniques, principles, and methodologies. From the definitions above, the term "**software engineering"** can be defined as the application of engineering principles to the design, development, and support of software. Software engineering is a field of engineering that deals with the design and development of software products. It works within a set of guidelines, principles, best practices, and methods. Software engineering is concerned with all aspects of software development, from system specification, design, and coding to system maintenance after it has been implemented. Software engineering can be described in terms of analysis and production.

THE NEED FOR SOFTWARE ENGINEERING: The demand for software engineering began when individual users, clients, firms, companies, and organizations starts to require a specific output from software they are using. They require developers to design and build software that will fit their day-to-day activities. This need was a result of several **challenges in software design and development**. The major challenge affecting software design and development is low-quality software products. This has led to the introduction of software engineering.

SOFTWARE ENGINEERING PROCESS: The first step in software engineering is to initiate a user request for a specific activity. The user's request comes from the head of the Information Technology (IT) Unit depending on the nature of the organization. The software development team upon receiving such a request, the project is broken down into different parts of user requirements.

IMPORTANCE OF SOFTWARE ENGINEERING: Software engineering is important because of the following reasons.

i. Software engineering ensures the creation of high-quality software products that are efficient, effective, and dependable.
ii. It ensures developers produce reliable and trustworthy systems in a cost-effective and timely manner.
iii. In the long run, it is usually less expensive to use software engineering methods and techniques for software systems rather than simply writing the programs as a personal programming project.
iv. It eliminates the high costs of software testing, quality assurance, and long-term maintenance cost.
v. It prevents the overhaul of the existing system because the majority of cost for most types of systems is the cost of changing the software after it has been installed.
vi. It is critical for organizations and businesses because it allows them to differentiate themselves from competitors and become more competitive.
vii. It can improve client experiences by introducing more features, client experience, and innovations.

CHARACTERISTICS OF GOOD SOFTWARE: The three main characteristics of good software include the following.

i. Operational characteristics

ii. Revision characteristics
iii. Transition characteristics

SOFTWARE EVOLUTION: The term software evolution was conceived by Lehman and Belady in 1976. It refers to the continuous development of a piece of software after the initial release to address changing requirements from the software end-user, firms, and market needs. It can also be referred to as the process by which a commercial computer program requires on going updating, maintenance, and improvement in order to remain a viable product over time. The primary goal of software evaluation is to **upgrade, migrate, and evolve currently existing software systems**. Software evolution ensures that commercial software systems are maintained and enhanced throughout their entire life cycle. The driving factor behind software evolution is the advancement in software development tools and other related technologies. Furthermore, there is a continuous change in business and consumer needs recently. The software developers and software engineering team are responsible for most of the processes of software evolution. This is because they design, develop, and produced software products used by organizations.

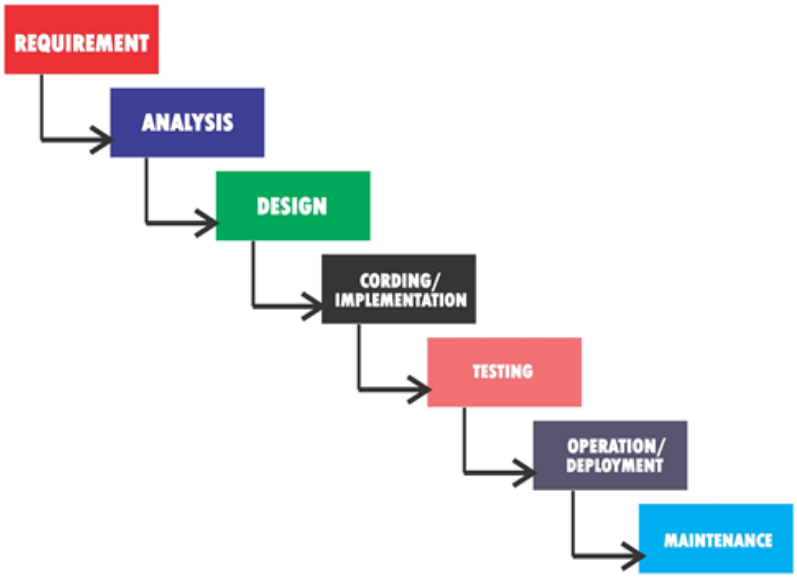
SOFTWARE PROCESS AND MODELS: A software process is a collection of related activities that result in the creation of software. It can also be referred to as the set of activities and associated outcomes that result in the production of a software product. These activities include creating new software from scratch or modifying existing software. These activities are usually performed by software engineers. There are four main activities of software processes that are common to all software processes. These activities include:

- **Software Specification:** It defines the main features of the software as well as the operational constraints that surround its functionality.
- **Software Implementation:** It ensures that software is designed and developed based on the user requirements identified. The process of turning a software design into a program is known as software implementation.
- **Software Validation:** It is the process of ensuring that the software system developed meets specifications and users' requirements. The validation process is important because software must adhere to its specifications and meet the needs of the customer. Software validation ensures the intended purpose of the software is met.
- **Software Evolution:** Also known as software maintenance, it is the process of changing, modifying, and updating software to meet client/end-user needs. Software evolution ensures that changes in the business environment and market requirements are met.

SOFTWARE PROCESS MODEL: A software process model is a simplified and specified representation of a software process. It is an abstraction of the process being described. A process model includes activities that are part of the software development process. Some examples of software process models are

i. Waterfall
ii. Spiral
iii. V-Model
iv. Rapid Application Development
v. Agile, etc.

WATERFALL MODEL: The waterfall model is a traditional model used in the system development life cycle to build a system in a linear and sequential manner. It is also known as a linear-sequential life cycle model. The model is referred to as a waterfall because it progresses systematically from one phase to another in a downward direction. The waterfall model divides project activities into linear, sequential phases. Each of the linear sequential phases is dependent on the deliverables of the previous phase and corresponds to task specialization. The output of one phase serves as the input for the next phase. Each phase must be completed before moving on to the next, and no phase will overlap another. The waterfall model is one of the least iterative and flexible approaches because progress is in one direction. This software development model is primarily used for small projects with no uncertain requirements.



Waterfall Model

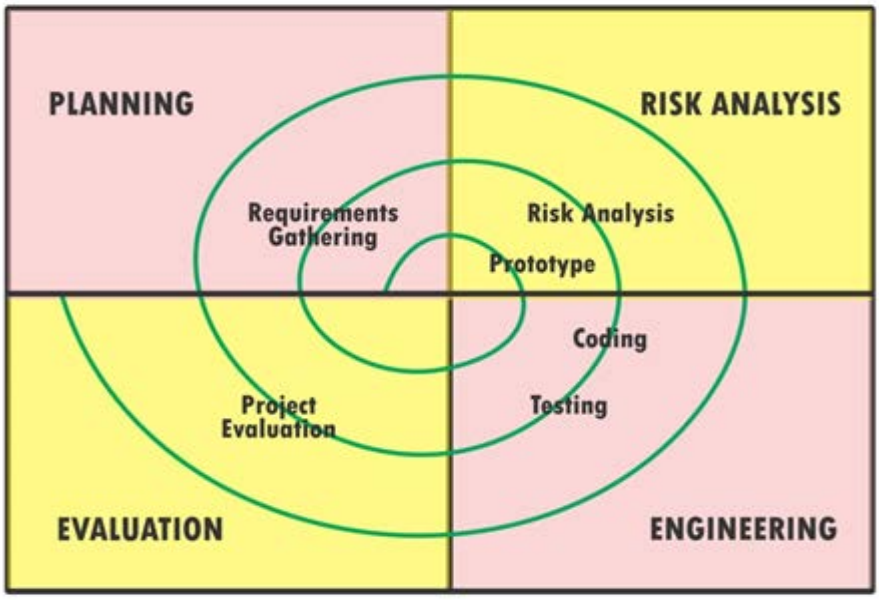
Advantages of the Waterfall Model

i. It is simple to understand.
ii. It is easy to use.
iii. It is a very rigid model.
iv. It reduces the number of problematic issues.
v. Transfer of information is very smooth.
vi. Project timelines are followed strictly.
vii. It works well for projects that are smaller in nature where requirements are well defined and understood.

Disadvantages of the Waterfall Model

i. It does not include user/client in the process.
ii. It has high amounts of risk and uncertainty.
iii. It delays testing until after project completion.
iv. It is not a good model for long and on-going projects.
v. Making Changes are difficult and can cause problem.

SPIRAL MODEL: The spiral model is a Systems Development Lifecycle (SDLC) method that combines the idea of an iterative development process with the systematic and controlled elements of a waterfall model. The spiral model places a strong emphasis on risk analysis, which is the most important feature of the model. It has the ability to manage unknown risks after the start of the project by creating a prototype system. Software engineers prefer the use of spiral models for large, expensive, and complicated projects because each phase of the spiral model begins with a design goal and ends with the client reviewing the progress.



Spiral Model

Advantages of the Spiral Model

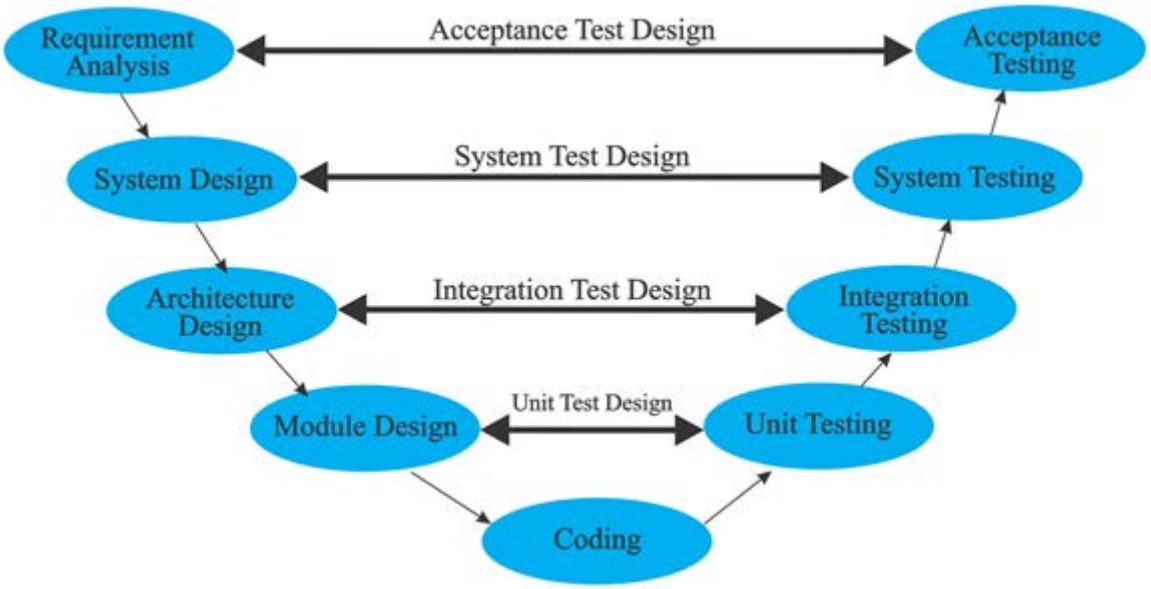
i. End-users get a first look at the system.
ii. Requirements are captured with accuracy.
iii. The spiral model facilitates customer feedback.
iv. Prototypes software products are used extensively.
v. Changes in requirements can easily be adopted and incorporated.
vi. The iterative development process facilitates effective risk management

Disadvantages of the Spiral Model

The following are disadvantages of the spiral model:

i. It is a difficult procedure.
ii. It is more difficult to manage.
iii. It is dependent on risk analysis.
iv. The spiral could go on indefinitely.
v. It is expensive for small projects.
vi. Not good for low-risk or small-scale projects.
vii. It is very difficult to determine the end of the project early.
viii. More documentation is required because it has intermediate phases.

V-MODEL: The V-model is the SDLC model where processes are executed sequentially in a V-shape. The V-Model is an extension of the waterfall model, which is based on assigning a testing phase to each development stage. The development process at each development cycle phase is directly related to the testing phase. The next phase of development begins only after the previous phase has been completed. There is a testing activity for each development phase. The V-model is also referred to as the Verification and Validation model. Figure below shows a V-model SDLC.



V-Model

Advantages of the V-Model

i. It is very easy to understand and simple to use.
ii. It is easy to manage because of its rigidity.
iii. It is suitable for small projects with well-defined requirements.
iv. It is a disciplined model in which phases are completed one at a time.
v. It supports accurate tracking of project progress by project management.

Disadvantages of the V-Model

i. It has high risk and uncertainty.
ii. It does not support iteration of phases.
iii. It is difficult to handle concurrent events.
iv. It is very difficult and expensive to make changes.
v. It is not suitable for long and on-going projects.

AGILE: The Agile SDLC model is a hybrid of iterative and incremental process models with a focus on process adaptability and client satisfaction through the rapid delivery of functional software. In the Agile model, each project must be handled differently, and existing methods must be tailored to best suit the project requirements. Tasks in the Agile Model are divided into time boxes or sprints, also known as small time frames, to deliver specific features for a release. The goal of sprints is to produce a working product at the end of each sprint. Every software product created after the iteration has more features than the previous one. The final software product developed contains all of the features required by the client. Figure below shows the Agile model SDLC.

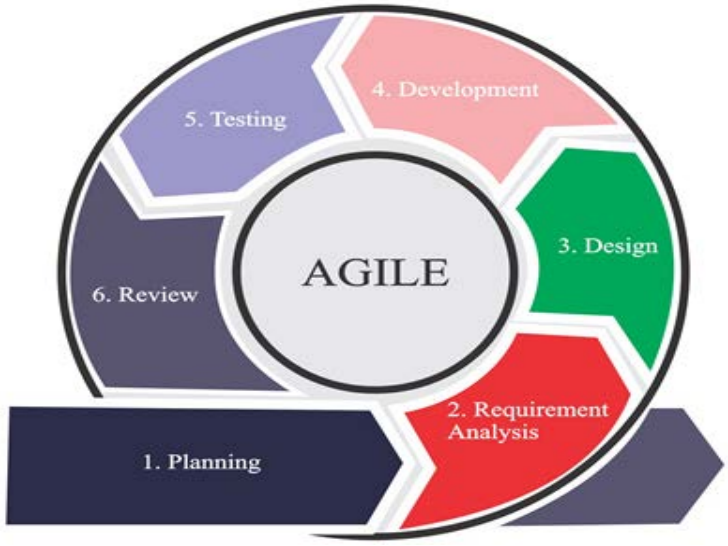


Figure 2.5: Agile Model

Advantages of the Agile Model

i. It requires small use of a resource
ii. It provides developers with flexibility.
iii. It encourages teamwork and cross-training.
iv. It is a practical approach to software development.
v. It is very easy to manage the development process.

Disadvantages of the Agile Model

i. It has very limited documentation.
ii. The model relies so much on individual.
iii. It has very poor planning of project resources.
iv. It is not good for dealing with complex dependencies.
v. There is a greater risk of sustainability, maintainability, and extensibility.

SOFTWARE PROTOTYPING: Software prototyping refers to the process of creating software similar to the final software product. A software prototype is created before the main and final software product is created. It is a demo version of the software program to be developed. A prototype product simulates how the final product will function but in most cases, only limited features and functionality are included. The prototype may not have the complete features and functionality as the final product. In addition, the prototype does not always contain the exact logic used in the final software. This is an extra effort that must be considered by the development team in their efforts of cost estimation in producing the final software product. Prototype product is created to allow software clients to evaluate and test developer proposals before they are implemented. It facilitates the collection of valuable customer feedback and assists the software development team in understanding the clients' expectations of the product under development.

Advantages of Software Prototyping

i. Missing functionality is easily identified.
ii. Difficult functions can easy be identified.
iii. It saves time and money by detecting defects much earlier.
iv. It enhanced user involvement in the product before its implementation.
v. Users can provide feedback more quickly, which leads to better solutions.

Disadvantages of Software Prototyping

i. Inadequate investigation on clients' requirement.
ii. Attachment to prototype by a developer
iii. Expense of implementing prototype
iv. Developer misunderstanding of client objectives
v. Clients may become confused between the prototypes and actual systems.

SOFTWARE REQUIREMENTS: Requirement engineering is the process of collecting, defining, validating, documenting, and maintaining the requirements in the engineering design process that are essential for the development of software to clients. Requirement engineering ensures the application of proven methods, principles, and tools to describe the expected behaviours and constraints of the proposed system. It is the task performed at the initial stages of the software development life cycle. Requirement engineering provides the necessary mechanism for determining what the client needs. The client's needs are received, assessed, and possible solutions are discussed with the team.

IMPORTANCE OF REQUIREMENT ENGINEERING: If the requirement engineering process is followed correctly, the final software to be developed will not have design and functionality challenges. This will results in successful software products for a client. The following are some of the importance of the requirement engineering process.

i. It provides the software developer with an idea of what the final software will be.
ii. It helps the software developer to define the scope of the software functionalities.
iii. It helps the software developer to determine the cost involve in developing the final software.
iv. It helps the software developer to determine the schedule to deliver software to the client.

REQUIREMENT ENGINEERING PROCESS: The requirement engineering process entails a series of activities that must be followed to collect all of the software project's specifications. Requirements engineering process consists of the following activities:

- **Inception**: The inception phase is where the business requirements are identified by the client and stakeholders. It is the phase where the concept of the software will emerge. A feasibility study of the business requirement is carryout and an analysis of the requirements is performed by the software developers. The scope of the software is determined and analysed at the inception phase between the client and the software developers. The features and functional requirements of the software are also discussed.
- **Elicitation**: Requirement elicitation is the process of finding and discovering detailed information on the software requirements from users, clients, and other stakeholders. Requirement elicitation ensures communication and discussion with key stakeholders to obtain information on the software project.
- **Specification**: The software requirement models are created through this activity. These models specify all requirements including functional and non-functional requirements, as well as constraints. The requirement specification in a software development project can be in the form of a written document, mathematical and graphical models, some usage scenarios, sort of code, sort of sample model, etc. Examples of the models used in this phase are ER diagrams, data flow diagrams (DFDs), data dictionaries, etc.
- **Negotiating**: Requirements negotiation refers to a discussion between the client and software engineer on requirements conflict to reconcile and negotiate with aim of satisfying both parties in the software project. Clients may add, reduce, remove and modify the requirements during conflict reconciliation and negotiation with the software engineer.
- **Validation**: Requirements validation is the process of ensuring that the development requirements define the software that the client truly desires. The nature of information gathered and comprehended during the engineering requirement process must be validated to ensure that all software requirements are specified. The software developer or engineer must ensure that the requirements are clear, accurate and consistent.
- **Management**: It is the process of tracking, analyzing, prioritizing, documenting, and agreeing on a requirement, as well as controlling communication with relevant stakeholders. Requirement management is a continuous process throughout a project. This is because software requirements keep changing throughout the software development life cycle.

TYPES OF REQUIREMENTS ENGINEERING PROCESS: The Requirements engineering process can be classified into three.

i. Business Requirement
ii. User Requirement
iii. Software Requirement

TECHNIQUES FOR REQUIREMENTS ELICITATION

Below are the most commonly used requirements elicitation techniques.

i. Stakeholder Analysis
ii. Brainstorming
iii. Interview
iv. Document Analysis/Review
v. Focus Group
vi. Interface Analysis
vii. Observation
viii. Prototyping
ix. Requirement Workshops
x. Survey/Questionnaire

DATA FLOW DIAGRAM: It is a visual depiction of information flows in a system. DFD uses known symbols to represent the processes of information flow within a system. It represents information using circles, rectangles, arrows, and short text labels to illustrate data inputs, processes, storage, output points and the paths between each information source and destination.

COMPONENTS OF DATA FLOW DIAGRAMS: The following are the four components of data flow diagrams using any convention's DFD recommendations.

i. External Entity
ii. Process
iii. Data Store
iv. Data Flow

SOFTWARE REQUIREMENT SPECIFICATION: Software Requirements Specification (SRS) is a document that explains in detail what the software will accomplish and how it will work. It also outlines the functionality that the product must have to meet the needs of all stakeholders. A good SRS document will define and describe in detail how the software will communicate with the user, other software and the system hardware components. It assists in making critical software product lifecycle decisions, such as when to remove a feature that is not necessary. The development phase became easier when a good time is taking in writing a detailed SRS. It helps the system developer to have a better idea of a software product, team, and the amount of time it will take to finish.

IMPORTANCE OF SOFTWARE REQUIREMENT SPECIFICATION: During the early stages of development, users and clients get a quick overview of the software. The following are some of the importance of SRS.

i. It serves as the foundation for the client-developer agreement.
ii. The goals, objectives and expected outcomes are clearly stated. Thus, it serves as a blueprint for software development.
iii. The targeted goals are established, which reduces the time and expense of the developers' efforts.
iv. It serves as a foundation for reviews.
v. It can be used as reference material at a later time.
vi. Because the software's basic functionality is specified, it becomes easier to transfer and use the solution elsewhere or with new customers.

QUALITIES OF A GOOD SOFTWARE REQUIREMENT SPECIFICATION

i. **Correctness**: The SRS should provide the correct scenario and not just provide a report for the sake of delivering one.
ii. **Unambiguous**: The SRS is the foundation of the agreement and can later be adjusted for reference and better understanding. As a result, it should include specific details about the scope, requirements, and results of the product being built.
iii. **Reliability**: The SRS should be trustworthy so that it may be passed on to the client's additional customers or other branches.
iv. **Verifiability**: The SRS should be verifiable, which means it should be a document that can be returned later on to double-check the workings of the product that has been built.

STRUCTURE OF SRS DOCUMENTS: This is a document that clearly outlines the format which can be used and should be considered to form a good software requirements specification. Depending on the type of demand, these requirements can be functional or non-functional. It is vital to thoroughly get the needs of customers; interaction between different customers and the software development team is carried out. The structure of the SRS document may not be the same for every software requirements specification. Below is an example of a software requirements specification format.

i. Cover Page
    a. Document Title
    b. Author(s)
    c. Affiliation
    d. Address
    e. Date
    f. Document Version (Optional)
ii. Introduction
    a. Purpose of the Document
    b. Scope of the Document

c. Overview
iii. General Description
iv. Intended Audience
v. User Needs
vi. Intended Use
vii. Definitions and Acronyms
viii. Assumptions and Dependencies
ix. Functional Requirements
x. Interface Requirements
xi. Performance Requirements
xii. Design Constraints
xiii. Non-Functional Attributes
xiv. Preliminary Schedule and Budget
xv. Appendices

SOFTWARE DESIGN PROCESS

SOFTWARE DESIGN: Software Design is a process of transforming user requirements into a form which can help the programmers to achieve the software development goals. It creates a software specification that assists in the coding and development of the software by the program developer. The activity that follows all required specifications before programming is referred to as software design. It encompasses all aspects of system conceptualization and implementation.

OBJECTIVES OF SOFTWARE DESIGN: The following are the objectives of software design.

i. **Correctness**: The software design should be correct according to the client's requirements. It must satisfy the requirements for the software.
ii. **Completeness**: The software design should include all components such as data structures, modules, and external interfaces, among others.
iii. **Efficiency**: The software design should make the program make good use of its resources.
iv. **Flexibility**: The design should be able to adaptable to changing needs.
v. **Consistency**: The design should be free of inconsistencies.
vi. **Maintainability**: The design should be simple enough that other designers may readily maintain it.

IMPORTANCE OF SOFTWARE DESIGN: The following are some of the importance of software design.

i. Simplicity: It makes the software easy to understand.
ii. Flexibility: Better-designed software ensures software adapts to changes when it occurs.
iii. Reusability: Software reuse is increased through well-designed software.
iv. Cost-efficiency: The cost-effectiveness of software is improved with good design.
v. Modularity: It breaks software into modules which makes it easy and convenient to work on software projects.
vi. Maintainability: It makes maintenance of software such as finding bugs and debugging much easier.
vii. Functionality: A good software design will reflect on how the software will work when is running.
viii. Performance: It reflects how well software and its component meets the requirements for timeliness.
ix. Portability: It gives the convenience of transferring functions from one module to another.

SOFTWARE DESIGN PROCESS: The design phase of software development is concerned with putting the client software requirement needs described in SRS documents into a form that can be implemented using a programming language. The software design process can be divided into three levels of design phases. The three levels of software design phases include the following.

- **Interface Design**: refers to the identification of the interaction between software and the environment it operates. At this level, the functions of the internal components of the systems are ignored. The system is considered an empty box. The dialogue between the target software and the users, devices, and other systems with which it interacts is the centre of attention.
- **Architectural Design**: The architectural design defines the essential structure and patterns of the system under development. It defines and specified the fundamental components of a software

system, such as the user interface, database, reporting module, and so on. Architecture design is more complex than detailed design.

- **Detailed Design**: The detailed software design defines the logical structure of each module as well as their interfaces to communicate with other modules. The detailed design focuses on all of the implementation details required to accomplish the stated architecture. It is more comprehensive and specific to modules and their implementations.

STAGES IN SOFTWARE DESIGN PROCESS: There are five (5) stages of the software design process.

Stage 1: Understanding Project Requirements

Stage 2: Research and Analysis

i. Interviews
ii. Focus groups
iii. Survey

Stage 3: Design

i. Data flow diagrams
ii. Technical Design
iii. User Interface

Stage 4: Prototyping

i. Low Fidelity Prototyping
ii. Medium Fidelity Prototyping
iii. High Fidelity Prototyping

Stage 5: Evaluation

SOFTWARE DESIGN TOOLS:

- **Jira**: This is a powerful software development tool that allows the software developer/designer to plan, monitor, release, and report on a project.
- **Mockflow**: It is a web-based wireframe tool that aids designers in planning, developing and sharing designs. It is a cloud-based tool and offers its users a large collection of mock components, icons, stickers, and so on.
- **Sketch**: This tool is utilized in the design of software user interfaces, mobile applications, websites, and icons. It provides all the tools required for a fully collaborative creative process.
- **Marvel**: Marvel delivers user-friendly design and prototyping tools that enable software developers to quickly wireframe, design, and prototype. It is a mobile-enabled cloud-based platform that allows single or multiple users and teams of varied sizes to create app prototypes in a centralized workspace.
- **Zeplin**: This is a design tool developer's use in creating front-end and user interfaces. It is a good platform for designers to organize, share, and interact with each other on design ideas.

UNIFIED MODELING LANGUAGE: The Unified Modeling Language (UML) is a general-purpose, developmental modelling language used in software engineering to provide a standard way to view system design. UML provides a standard way to visualize architectural plans for a system, including elements such as tasks, actors, business operations, application frameworks, components, programming language statements, and reusable software modules.

Structure diagrams Structure diagrams emphasize what things must be in the system being modelled

1. **Class diagram:** describes the structure of a system by showing the system's classes, their attributes, and the relationships among the classes.
2. **Component diagram:** depicts how a software system is split up into components and shows the dependencies among these components.
3. **Composite structure diagram:** describes the internal structure of a class and the collaborations that this structure makes possible.
4. **Deployment diagram** serves to model the hardware used in system implementations, the components deployed on the hardware, and the associations among those components.
5. **Object diagram:** shows a complete or partial view of the structure of a modelled system at a specific time.

6. **Package diagram:** depicts how a system is split up into logical groupings by showing the dependencies among these groupings.

Behavior diagrams Behavior diagrams emphasize what must happen in the system being modelled

1. **Activity diagram:** represents the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.
2. **State diagram:** standardized notation to describe many systems, from computer programs to business processes.
3. **Use case diagram:** shows the functionality provided by a system in terms of actors, their goals represented as use cases, and any dependencies among those use cases.

Interaction diagrams control flow of control and data among the things in the system being modelled:

1. **Communication diagram** shows the interactions between objects or parts in terms of sequenced messages. They represent a combination of information taken from Class, Sequence, and Use Case Diagrams describing both the static structure and dynamic behavior of a system.
2. **Interaction overview diagram:** a type of activity diagram in which the nodes represent interaction diagrams.
3. **Sequence diagram**: shows how objects communicate with each other in terms of a sequence of messages. Also indicates the lifespans of objects relative to those messages.
4. **Timing diagrams:** are a specific type of interaction diagram, where the focus is on timing constraints.

MARKUP LANGUAGES AND DESIGN IMPLEMENTATION: Extensible Mark-up Language (XML) is a mark-up language and file format that can be used to store, transmit, and reconstruct arbitrary data. The Extensible Markup Language (XML) is a simple text-based format for representing structured information such as documents, data, configuration, books, transactions, and invoices, among other things. XML is the most popularly used formats for sharing structured information between computers, programs and people these days. XML is also use to share information within Local Area Network and across networks.

Rules for Extensible Markup Language

i. It must start with an XML declaration.
ii. It must have a single distinct root element.
iii. All XML document start tags must match end tags.
iv. XML tags are case sensitive.
v. Every element must be closed.
vi. All elements must be nested properly.
vii. The values of all attributes must be quoted.
viii. For special characters, XML entities must be used.

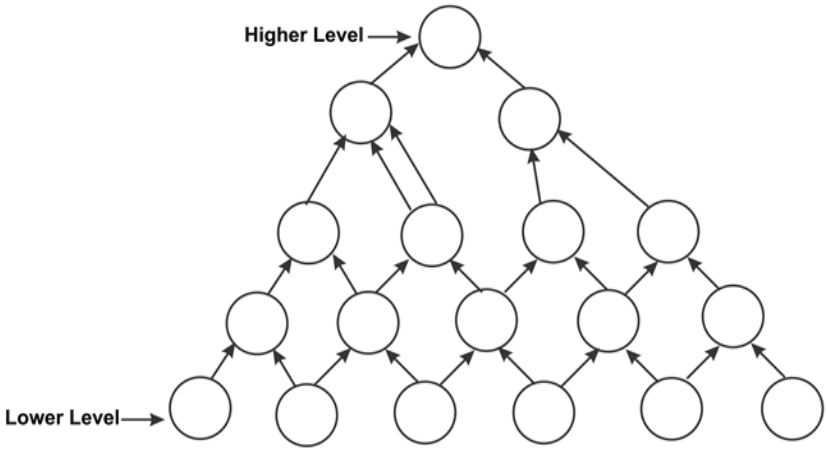
Advantages of Extensible Markup Language

i. Redundancy: XML markup is extremely expressed in more words. For instance, every end tag, such as /description>, must be provided. This allows the computer to detect common errors like incorrect nesting.
ii. Self-describing: the readability of XML as a text-based format and the presence of entities and attribute names, individuals looking at an XML document can often get a good start on understanding the format and it also helps users seek mistakes.
iii. Network effect and the XML Promise: XML tools will be able to read and process any XML document.
iv. XML separates data from HTML: When there is a need to display dynamic data in an HTML document, editing the HTML code each time the data changes will be a time-consuming process. Data can be stored in separate XML files using XML.
v. XML simplifies data sharing: most computer systems and database applications these days contain data in inconsistent file format. Plain text is used to store XML data.

SOFTWARE DESIGN STRATEGIES: Software design strategies refer to the use of system design methodologies to conceptualize software requirements. Below are two software design strategies. They are as follows.

i. Bottom-up Approach
ii. Top-down Approach

- **Bottom-up Approach**: The design process begins with the most basic components and subsystems. It prioritizes the design of subsystems and the lowest-level components. These components are used to create the next higher-level component and their sub systems. Higher-level subsystems and larger components can be built more easily and efficiently if these components are designed first. The process is repeated until all of the components and subsystems have been combined into a single component, which is referred to as the complete system. As the design progresses to the next higher levels, the amount of abstraction increases. The process of assembling lower-level components into larger sets is repeated until the entire system is composed of a single component. This design strategy also makes generic solutions and low-level implementations more reusable. The bottom-up design approach is ideal when creating new software. Figure below shows the bottom-up strategy of a software design strategies.



Bottom-up Approach

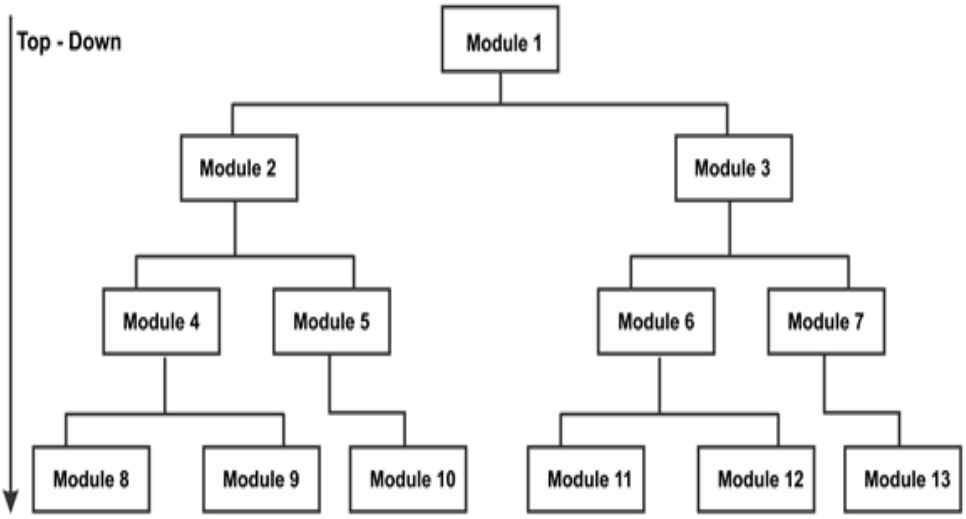
Advantages of Bottom-up Approach

i. Early on, advantages are realized.
ii. The benefits are much when general solutions can be reused.
iii. It can be used to conceal low-level implementation details.
iv. It can be combined with the top-down approach.
v. There is no need to create custom adapters in the early stages.
vi. Several more manual processes can be replaced with early automation.

Disadvantages of Bottom-up Approach

i. It has little to do with the problem's structure.
ii. High-quality bottom-up solutions are extremely difficult to create.
iii. Instead of business processes, this strategy is driven by existing infrastructure.
iv. This strategy is moved by the existing infrastructure instead of the business processes.
v. It results in an overabundance of 'potentially useful' functions rather than the most appropriate ones.

- **Top: Down Approach:** This design strategy is totally focused on first dividing the software into subsystems and components. Rather than developing from the bottom up, as in the bottom-up strategy, the top-down design strategy conceptualizes the entire system first and then divides it into several subsystems. These subsystems are further designed and subdivided into smaller subsystems and sets of modules that meet the larger system's requirements. The entire software system is regarded as a single entity. The system is divided into sub-systems and components based on their characteristics. The same is true for each sub-system. This process is repeated until the software's lowest level is reached. The software design process begins with defining the system as a whole and moves on to define the subsystems and components. Top-down design is better suited for software solutions that need to be developed from the ground up. Figure 4.3 below shows the top-down strategy of a software design strategies.



Top-down Approach

Advantages of Top-down Approach

i. The first implementation serves as a demonstration of the identity management solution.
ii. Its strong emphasis on requirements aids in making a design responsive to its requirements.
iii. The primary effect on the operational and maintenance resources is not extreme as the bottom-up approach.
iv. The organization realizes a more focused use of resources from the individual managed application.
v. When the phases for the managed application are completed, a deeper, more mature implementation of the identity management solution is required.

Disadvantages of Top-down Approach

i. In the early stages, the solution provides limited coverage.
ii. There is a need to create custom adapters at an early stage.
iii. The cost of implementation is likely to be higher.
iv. Project and system boundaries are typically software specific oriented. As a result, the benefits of component reuse are more likely to be missed.
v. The system is likely to overlook the advantages of a well-structured and simple architecture.

SOFTWARE DEVELOPMENT: Software development is a collection of distinct tasks. These tasks involve understanding the problem and defining requirements, designing the solution, writing the code, and debugging it. Unfortunately, the way people think about software development is out-dated. Currently, software development must be considered as a sequence of distinct steps with numerous continuous procedures that guide software development and ensure quality. Software engineering is commonly thought of as a collection of different, discrete operations (design, coding, testing) that culminate in a completed product.

ACTIVITIES INVOLVED IN SOFTWARE DEVELOPMENT: Consider the table below. The top row is made up of separate activities each with its own set of entry and exit criteria. The lower rows reflect on-going activities, or tasks that should be completed throughout the life of the software project.

Discrete activities with some sample continuous activities

Analysis	Design	Implementation	Test	Maintenance
Plan for maintenance				
Software Quality Assurance				
Verification and validation				
Risk analysis and risk management				
Configuration management				
Software engineering project management				

APPLICATION PROGRAM INTERFACE (APIs) AND USES: An API is a set of operations that allow applications to access data and connect with external software components, operating systems, or microservices. APIs are powerful tools that can help you speed up business procedures; expand your brand's reach, link buyers with the products they desire, and more. An API transmits a user response to a system and then receives the system's answer. API requests can be done in four different ways:

a. GET-Gathers information (Pulling all Coupon Codes)
b. PUT-Updates pieces of data (Updating Product pricing)
c. POST-Creates (Creating a new Product Category)
d. DELETE- (Deleting a blog post)

JavaScript Object Notation (JSON) and used

On a server, data is represented using JavaScript Object Notation (JSON). Humans can read it easily, and machines and programs can understand it as well. An example of JSON from the BigCommerce product:

```
{“name”: “BigCommerce T-Shirt”,
        “price”: *25.00”,
        “Category”: “Shirts”,
        ],
        “weight”:  4’
        “type”: “Physical”
}
```

This is simple to comprehend because it is presented as key/value pairs, with the key on the left and the value on the right. Keys are a fixed objects defined by the program that will remain the same as a category, however values, such as "Shirts," will be unique.

USES OF API: APIs assist developers in delivering data to customers, from online purchasing to reading a social media app to playing a smartphone game. API is used every time someone visits a webpage on the internet.

i. Visiting a bank: Imagine yourself as a user, a bank teller as an API, and the bank as the system with which you interact.
ii. Creating a Facebook account: If you put in “John Smith” on Facebook, the API alerts Facebook’s servers that you are looking for John Smith. Facebook then provides you a list of all the people who have the same name as you.
iii. Keeping up with current events on social media: open Twitter and go to the 'Sports' section. The Twitter API makes it simple to view a variety of tweets relating to the desired team winning the competition.

Other types of API use

a) Open APIs are freely accessible to the public.
b) Companies create partner APIs to provide API access to strategic business partners as an additional revenue stream for both parties.
c) Private APIs is intended for internal usage only and is not intended for public use.

FACTORS TO CONSIDER FOR SOFTWARE DEVELOPMENT TOOL

i. **Usefulness:** The fundamental consideration when determining whether or not to employ a certain tool, and the implementation of that tool to use, is the value it will bring to the project's overall completion.
ii. **Environment Applicability:** Not all tools are appropriate for all situations. A web deployment tool, for example, will be useless for a Windows desktop program.
iii. **Company Standards:** In larger organizations, and often in smaller ones as well, the use of specific tools will be required in order to meet objectives or adhere to set policies. Standardization of tools can make it easier for an organization to move developers between projects as needed, as well as provide confidence to management that identical processes are followed throughout projects and project teams, resulting in consistent product quality.
iv. **Prior Team Tool Experience:** Almost all software has a learning curve to some degree. The level of experience that developers have with various tools can influence the tools they choose.

That specific experience can also be utilized to determine whether a tool will be effective in the project since developers are known for having strong opinions on such topics and are not afraid to share them.

v. **Integration:** A tool's ability to integrate with other tools has a significant impact on the value it provides to the team and the project. Some integration is done for the sake of "convenience." Other deeper integration combines data and reacts to events across tools to provide high value to the team and other groups inside the organization.

vi. **Overhead:** Complex tools might take time and effort to implement with a team and integrate with existing development software. Many tools require time and effort to use, in addition to initial implementation and learning curve. When assessing the tool's overall value, this overhead should be considered.

TYPES OF SOFTWARE DEVELOPMENT TOOLS AND USES : A number of tools are available to help in the software development process. The following are some of the most well-known tool categories:

i. Java is one of the most well-known programming languages. It is employed in higher education institutions and huge corporations.

ii. Python is a relatively user-friendly language that many novices like. Python's syntax is basic and straightforward, which is why so many newcomers choose it.

iii. Ruby: Like Python, Ruby is known for being a beginner-friendly language. It's simple to read, and it was created with the idea that programming should be enjoyable.

iv. JavaScript: This is a text-based web development language. All websites require JavaScript to function.

v. C: This is the parent language because it is one of the first programming languages. Although C is notorious for being difficult to learn, it is ideal for high-performance applications.

vi. C++: this language is based on C and extends it with new features. It's also an older one with a reputation for being difficult to master. C++ is still taught in several colleges.

PROCESS OF DEVELOPING SOFTWARE

The Client, Developer, and User are all present in any software development project. Development activities vary per organization, but they often include:

a. Development-all phases of software development
b. Maintenance-revisiting the developed software

PHASES OF SOFTWARE LIFE-CYCLE:

a) **Requirements:** The client approaches the developer with their issue. Following the client-developer interaction, the developer determines exactly what the customer wants, and defines restrictions (deadline, cost, reliability, security, etc.). These needs are both functional and non-functional requirements. Functionalities are refined and evaluated in subsequent meetings in terms of feasibility (called systems analysis).

b) **Specification:** When the user is pleased with the requirement, the developer creates a specification document - WHAT to do, HOW to accomplish it, and WHEN to do it. Functionality; precisely what the system is expected to perform, Constraints must be satisfied, and Input/output are all spelled out in the specification (report). The contract is represented by specifications.

c) **Design:** A significant step toward meeting the specifications: The internal structure of the product is determined here, and the algorithm + data structures are chosen. Determines the flow of internal data and Breaks the product down into modules/subsystems, which are self-contained bits of code with well-defined interfaces. The following are the stages of system design:

   i. Data design: In this phase, the data structure is created.
   ii. Architectural design-the structural units are created during this phase (classes)
   iii. Interface design-the interfaces between the components are defined in this phase.
   i. In procedural design-in, this phase, the algorithms for each procedure are specified.

d) **Implementation:** Programmers code the various subsystems. Remember the structured Programming Concept! The main document here is the source codes, which are fully commented on.

e) **Integration:** Put the subsystems together to see if the product works properly as a whole. It could happen all at once or one by one. Stubs and Drivers may be required—to be discarded at the conclusion of the process.
f) **Maintenance:** Maintenance is an important aspect of the software life cycle. After the client has accepted the software product, all updates after the warranty period are considered maintenance.
g) **Retirement:** When continued maintenance is no longer cost-effective, such as when planned changes in requirement are so dramatic that the design itself must be changed. Because of the changes in hardware and the new OS, constructing a new system is less expensive than maintaining an existing one.

PROGRAMMING PROCESS, STANDARDS AND PROCEDURES: Codes must be written in a way that is understandable not only to the developer (programmer) when revisiting the codes for testing but also to others as the system evolves. For example, the people that may want to reuse the codes in near future. Advantages must be taken from the characteristics of the design's organization, data structure, and the programming language construct while still creating easily reusable code. There are general guidelines that will shield the developer from some shortcomings. Thus the programming standards and procedures that should be followed are discussed below:

PROGRAMMING PRINCIPLES

A. **Class Design**

i. The Single Responsibility Principle: There should be only one cause for a class to change.
ii. The Open-Closed Principle: You should be able to expand the behavior of a class without having to change it.
iii. The Liskov Substitution Principle: Functions that employ base class pointers or references must be able to access derived class objects without being aware of them.
iv. The Interface Segregation Principle: Create fine-grained, client-specific interfaces, with no client being compelled to rely on techniques it doesn't employ.
v. The Dependency Inversion Principle states that you should rely on abstractions rather than concretions.

The Cohesion of the Package

i. The Release Reuse Equivalency Principle: "Either all of the classes inside the package are reusable, or none of them are."
ii. The Principle of Common Closure: Classes that change at the same time are packaged together.
iii. The Principle of Common Reuse: Classes that are frequently used together are bundled together.

Package Decoupling: Packages can be decoupled by splitting their responsibilities and turned into releasable packages like simple software libraries.

i. The Acyclic Dependencies Principle: The package dependency network must have no cycles, which means that each component that can be detached from the rest of the system should be developed and built separately.
ii. The Principle of Stable Dependencies: Depend on the direction of stability. Only depend on packages that are more stable than you are.
iii. The Principle of Stable Abstractions: Abstractness rises as stability rises.

Code Comment

i. Comments do not make up for bad code.
ii. Explain yourself in code.
iii. Do not add noise comments e.g.

```
/** The day of the month. */
 private int dayOfMonth;
```

/**
* Returns the day of the month. *

iv. Do not express your emotions in comment, talk to your supervisor.

v. Follow comment conventions. Depending on language you are using, follow common comment conventions.

Many programming languages employ the same comment style.

PROGRAMMING GUIDELINES:

- **Control Structures**: Many control structures are recommended during architectural design; it is, therefore, necessary to preserve these control structures as designs are converted to codes, using architectures such as implicit declaration and Object-Oriented Design.
- **Modularity:** A good design must be modular. Modularity is the process of breaking down a complex problem into smaller, more manageable components, each of which is independent of the others. The modularity of codes is significant because it hides implementation details; it also makes coding easier, allows for a quick understanding of codes, and allows for code maintenance and reuse.
- **Generality:** The concept of generality is born out of the necessity for flexibility. Cohesion and coupling, that is, the interaction between parts in a program and their level of dependencies, are two crucial notions here. Because this has a significant impact on programming, parameter names and comments are required to demonstrate connectivity between program components.
- **Data structures:** That is, strategies for structuring data that can be processed by computers. Although some programs specify their own data structures for execution, data rearrangement makes program calculations easier. When it comes to data structures, recursively refers to a data structure's ability to recognize the first element and construct the second one using previous data.

SOFTWARE TESTING AND ITS IMPORTANCE: It's vital to remember that testing uncovers about half of all errors, according to industry statistics. The rest can only be discovered by using the provided program. The focus of the testing phase should be on employing testing to eliminate root causes of problems to ensure strong customer relations and deliver quality software. One common mistake made in large projects is to reduce testing time as the project deadline approaches. The mistake reasons are identified, corrected, and eliminated throughout the testing phase.

Attributes of a good test

i. A good test has a high chance of detecting an error. To accomplish this, the tester must first comprehend the software and then attempt to visualize how it can fail. The failure classes should ideally be investigated.

ii. A decent test is not pointless. Time and resources for testing are limited. It's pointless to run a test that serves the same function as another. Each test should serve a distinct objective (even if it is subtly different).

iii. A good test should be neither too easy nor too difficult to understand. Although it is occasionally possible to integrate multiple tests into a single test case, the potential side effects of this method may obscure problems.

SOFTWARE TESTING PHASES

a. **Unit Testing:** Unit testing is important control pathways that are checked using the component-level design description as a reference to find faults within the module's boundary. The narrow scope defined for unit testing limits the relative complexity of tests and uncovers mistakes. The developers of a unit are often too close to the code to spot obvious problems. Furthermore, they have an unconscious hope that the unit will not fail during testing, which influences their test case selection.

b. **Integration Testing**: After each device has been tested, the interfaces between units must be tested. This is the most important aspect of the testing process. While unit testing examines separate modules, integration testing examines how they interact. Inter developer communication is tough in complicated systems because there are often several developers.

c. **Validation Testing**: Integration testing is completed when the program is fully built as a package, interfacing issues have been discovered and repaired, and a final series of software tests-validation testing can begin. Validation can be characterized in a variety of ways, but a basic definition is that validation is successful when software performs as the client would reasonably expect.

d. **System Testing**: System testing is a collection of tests with the primary goal of completely exercising a computer-based system. Even though each test has a different goal, they all aim to ensure that system pieces have been correctly integrated and are performing their assigned roles. Assuming that unit and integration testing went well, the next step is to bring the entire system together and test its functionality. This form of testing isn't meant to determine if the units or interfaces operate; rather, it's meant to demonstrate that the system fits the requirements.

SOFTWARE TESTING METHODS

a. **White-Box Testing:** White-box testing, also known as glass-box testing is a test case design process that derives test cases from the procedural design's control structure. The software engineer can create test cases that (1) ensure that all independent paths within a module have been tested at least once, (2) exercise all logical decisions on their true and false sides, and (3) execute all loops at their boundaries and within their operational bounds, and (4) exercise internal data structures to ensure their validity using white-box testing methods. Unit testers have access to the unit's real code once again. Before preparing test cases, testers might design their tests by looking over the code. Branch-coverage testing, statement testing, loop testing, and coverage testing are examples of common white-box tests. White-box testing is required to ensure that the code performs as expected. Furthermore, white-box testing shows that no additional code is there that is doing tasks that are not required. This "added code" could be erroneous or dangerous.
b. **Black-Box Testing**: Black-box testing, also known as behavioral testing, focuses on the software's functional requirements. In other words, black-box testing allows a software engineer to create sets of input conditions that fully exercise all of a program's functional requirements. Black-box testing is not a replacement for white-box testing. Rather, it's a supplement to white-box approaches, and it's more likely to find a different set of problems. Black-box testing looks for problems in the following areas: (1) improper or missing functions, (2) interface errors, (3) data structure or external database access errors, (4) behavior or performance errors, and (5) initialization and termination errors are all examples of errors. Unlike white-box testing, which is done early in the testing process, black-box testing is done later. Because black-box testing ignores control structure on purpose, attention is drawn to the information domain.
c. **Static Testing**: Static testing refers to inspecting code with tools rather than running it. This type of testing should be performed during unit testing to see whether all variables have been declared and initialized or if they have never been used. Most languages have specialized tools that can perform a variety of code checks. On a project, the use of static test tools should be standardized and enforced.

SOFTWARE TESTING TYPES

a. **Recovery testing**: This is a system test that causes the software to fail in many ways and checks for a good recovery. Re-initialization, check-pointing mechanisms, data recovery, and restart are all checked for correctness if recovery is automatic (done by the system itself). If human intervention is required, the mean-time-to-repair (MTTR) is assessed to see if it is within acceptable parameters. The goal is to see how the system reacts to unforeseen situations.
b. **Security Testing: S**ecurity testing verifies that a system's built-in protective mechanisms will, protect it from improper penetration. During security testing, the tester takes on the role(s) of someone who wants to get into the system.
c. **Stress Testing:** Stress testing is used to expose the software to unusual scenarios. In other words, the stress tester asks, "How high can we turn this up before it fails?" Stress testing involves running a system in a way that places unusual demands on resources in terms of quantity, frequency, or volume.
d. **Performance Testing:** Performance testing is used to evaluate software's run-time performance in the context of a larger system. All stages of the testing process include performance testing. Even at the unit level, white-box tests can be used to examine the performance of specific modules.
e. **Dynamic Testing:** Tests that investigate the state of your software during a sample run are referred to as dynamic testing. It analyses the execution paths that a run takes as well as the memory utilization throughout that run. If at all possible, it is usually done during unit testing.

f. **Acceptance Testing:** The acceptance test, as discussed in systems testing, is a formal test that shows the user that the system meets all of their requirements. The criteria for an acceptance test will be employed as part of the system test in a good project. If a project passes the system test but fails the acceptance test, the system testing was ineffective.

SOFTWARE PROJECT MANAGEMENT: The art and science of organizing and supervising software projects are known as software project management. It is a sub-discipline of software project management that involves the planning, implementation, monitoring, and control of software projects. It is a method of organizing, assigning, and scheduling resources to create computer software that meets the criteria. The client and the developers need to know the project's length, duration, and cost in software project management. Software project management has three requirements. These are: ***Time, Cost*** and ***Quality***. It is an important aspect of the software development process to provide a high-quality product while staying within the client's budget and on time. Various external and internal factors may have an impact on this triple factor. Any of the three factors have a significant impact on the other two.

- **Software Project Management:** A software project is the entire process of software development, from requirement collecting to testing and maintenance that is carried out in a specific time frame to produce the desired software product, according to the execution techniques.
- **Project manager:** A project manager is a figure who is responsible for the overall planning, design, execution, monitoring, controlling, and closure of a project. A project manager is a character who is in charge of making major and minor project choices. The project manager is in charge of managing risk and reducing uncertainty. Every decision made by the project manager must benefit the project.

ROLE OF A PROJECT MANAGER

a. **Team leader**: A project manager must lead his team and guide them so that everyone knows what is expected of them.

b. **Middleman:** The project manager serves as a go-between for his clients and his staff. He is responsible for coordinating and transferring all pertinent information from clients to his team, as well as reporting to senior management.

c. **Mentor:** He should be present to lead his team through each step and ensure that they are connected. He gives his team a recommendation and properly directs them.

RESPONSIBILITIES OF A PROJECT MANAGER

a. Managing difficulties and hazards
b. Form a project team and allocate tasks to various members.
c. Activity sequencing and planning
d. Progress tracking and reporting
e. Adjusts the project plan to account for the situation.

Project Proposal: A project proposal is a project management document that outlines the project's goals and requirements. It aids in the agreement of an initial project planning framework between companies and external project stakeholders. The purpose of proposing is to gain support for a project and to bring a vision to life. The proposal must communicate to the audience and then motivate them to move forward with the project. The steps are listed below:

Step 1: Identify the issue. What problem is the project attempting to solve? What exactly is the issue? Why is it important to solve it? Make the audience perceive the problem through the eyes of the author.

Step 2: Present the answer. How will the project address the issue? Why is this solution superior to others that are similar? Discuss why other options aren't feasible in this case.

Step 3: Create a list of deliverables and success criteria. This includes functions and attributes, and limit for supply.

Step 4: Clearly state the plan. This is the most important element of the proposal because it explains how to meet the project's goals. It also explains how issues will be dealt with.

Step 5: Create a timetable and a budget. This is where you break down project expenditures and explain how you'll fulfil deadlines.

Step 6: Bring everything together. Finish your proposal with a quick summary of the problem, solution, and advantages. Make your proposal stand out by restating concepts or information you want the audience to remember.

Step 7: Make changes to your proposal. Rewrite the proposal to make it more engaging, useful, clear, and convincing.

PROJECT MANAGEMENT TOOLS

a. Gantt graph: Henry Gantt created the Gantt chart (1917). It depicts the project schedule in terms of the time intervals. It's a horizontal bar chart with bars reflecting project activities and time allocated to them.

b. PERT Chart: A PERT chart (Program Evaluation and Review Technique) illustrates a project as a network diagram. It can graphically display the project's essential events in both parallel and sequential order. When events occur one after another, the subsequent event is dependent on the previous one. Numbered nodes represent events. They're connected by labelled arrows that show the project's task order.

c. Resource Histogram: This is a graphical tool that shows the number of resources (typically experienced employees) required for a project event across time as a bar or chart (or phase).

d. Critical Path Analysis: This tool is excellent for identifying project interdependencies. It also aids in determining the shortest or most critical path to properly complete the project. Each event is given a specified time range, similar to a PERT diagram. The events are listed in order of the earliest start time feasible. The path between the start and finish nodes is a crucial path that cannot be decreased further, and all events must be performed in the same order.

ACTIVITIES OF SOFTWARE QUALITY MANAGEMENT:

a. **Software Quality Assurance** (SQA): This is a set of tasks for ensuring software engineering procedures are of high quality. SQA ensures that the software development process progresses to the next phase only when the current/previous phase meets the needed quality criteria. The following activities are included in SQA:

   i. Definition and execution of processes
   ii. Auditing and
   iii. training

   The following are the responsibilities of Quality Assurance:

   i. Identifying process flaws
   ii. Address those flaws to keep the process improvement.

FACTORS AFFECTING SOFTWARE QUALITY AND DEVELOPER PRODUCTIVITY

a. Individual ability
b. Systematic approach
c. Change control
d. Level of technology
e. Level of reliability
f. Available time
g. Required skills
h. Adequacy of training

THE PROCESS OF SOFTWARE VALIDATION AND VERIFICATION

- **Verification:** Verification provides an answer to the inquiry, "Are we making the software correctly?" The first stage in verification, then, is to create and follow a procedure for the software project. Verification can be accomplished by having only technical or in-house staff attend, and validation can be accomplished by having users or user representatives attend. It's crucial to note that evaluations begin with the requirements analysis step. The organization should establish checklists, and individuals should update them as needed.
- **Validation:** Validation is concerned with the question of whether or not we are developed meets the user's requirements. High-quality software is useless if it does not meet the needs of the consumers.

Representation of the following relations among classes using UML diagram

i. *Students credit 5 courses each semester. Each course is taught by one or more teachers.*

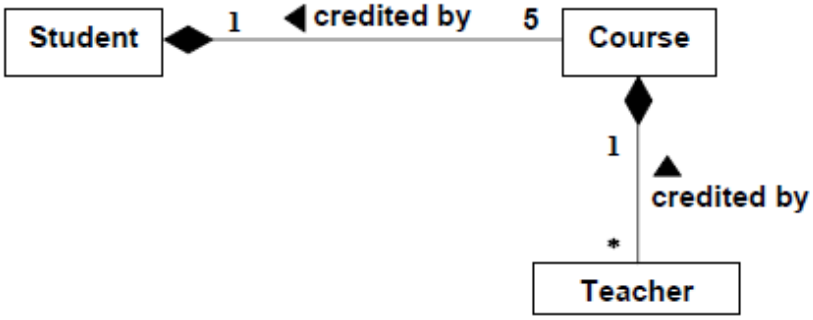
ii. *Bill contains number of items. Each item describes some commodity, the price of unit, and total on this price.*

iii. *An order consists of one or more order items. Each order item contains the name of the item, its quantity and the date by which it is required.*

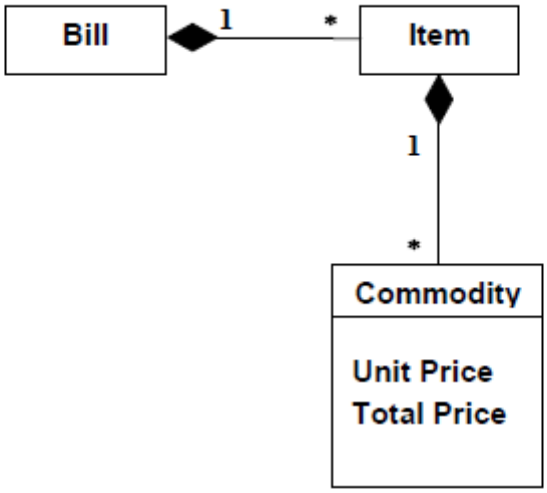
Each order item is described by an item type specification object having details such as its vendor addresses, its unit price, and the manufacturer.

Ans.: -

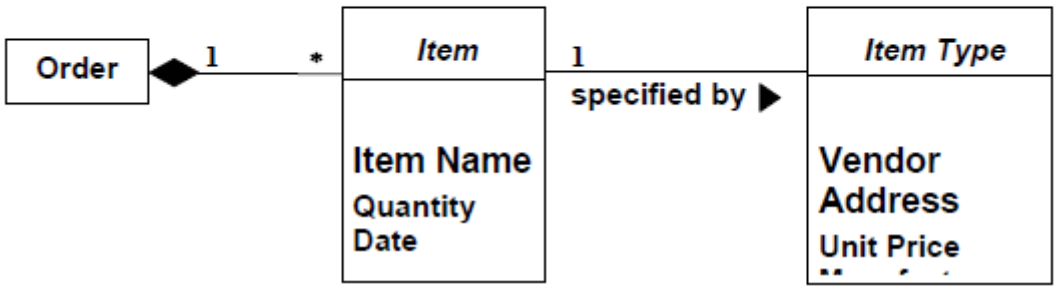
1)



2)



3)



SOME USEFUL SELF-STUDY QUESTIONS

1. Define software engineering
2. What is the need for software engineering
3. Briefly explain the software engineering process
4. List any five (5) members of the IT Management team in an Organization
5. Enumerate any five (5) importance of software engineering
6. List and explain the characteristics of good quality software
7. Briefly Explain what you understand by software evolution
8. Define a Model
9. Regarding *On-site customer* practice in XP as in line with challenges faced, states Three (3) advantages, Three (3) characteristics and Three (3) roles and responsibilities of a good customer
10. Having a Product Management Team (PMT) can reduce the problem of *"On-site customer"* Practice's problems, why? Or why not?
11. Why changing life cycle model between releases as product matures from either Water fall, Rapid Prototype and Commercial Off-The-Shelf (COTS). States the reasons Why this is so?
12. Explain the ***requirement***t & ***Design*** phases of software life cycle. Be careful to clearly explain the roles played by the ***client***, ***developer*** and the ***user***.
13. Differentiate between validation and verification
14. Software Development activities vary from one organization to another, but broadly speaking they cover development and maintenance, hence states all the phases of software life cycle and test.
15. Identified five (5) types of requirements.
16. Why testing software? Hence differentiate between defect testing and debugging
17. Briefly explain five testing strategies you know?
18. Briefly explain what you understand by a software process
19. What are the four (4) main activities of software processes that are common to all software processes?
20. With example, briefly explain the software process model
21. States five (5) uses, advantages and disadvantages of the Waterfall Model
22. With the aid of a diagram, explain the Agile SDLC model
23. Define Software Prototyping
24. State and explain the types of software prototypes that are commonly used in an industry
25. Define requirement engineering
26. State five (5) importance of requirement engineering
27. With the aid of a diagram, explain the requirement engineering process
28. List and explain the types of the requirements engineering process
29. State the three (3) types of software requirements
30. Elicitation is a crucial part of any software project because it is responsible for gathering software requirements. States the most commonly used requirements elicitation techniques you know
31. States and explain the components of Data flow diagrams
32. What are the rules and tips for using Data flow diagrams?
33. With the diagram, briefly differentiate the three (3) levels in the data flow diagram.
34. Write short notes on the following stages in the software design process

(a.). Understanding Project Requirements (b). Research and Analysis, (c). Design, (d). Prototyping, (e). Evaluation

35. State any five (5) most common software design tools you know
36. With the aid of a diagram, explicitly distinguish between the bottom-up and top-down approaches of software design strategies
37. States five (5) merits and demerits of the bottom-up approach of software design strategies.
38. Explain why is it necessary to create a model in the context of good software
39. Codes must be written in a way that is understandable not only to the developer (programmer) when revisiting the codes for testing but also to others as the system evolves. Discuss.
40. Explain what you understand by Package Decoupling.
41. List and discussed the Programming Standards and Procedures you know.
42. Highlight the important of Code Reuse in Future Project
43. List and briefly explain Attributes of a good software testing
44. Explain what you understand by the words software prototype?

45. Identify four characteristics of a good software design technique.
46. Define the term cohesion in the context of object-oriented design.
47. The art and science of organizing and supervising software projects are known as software project management. Discuss.
48. Explain who is a Project manager. Hence itemise the role of a project manager.
49. A project proposal is a project management document that outlines the project's goals and requirements. List the steps of project proposal.
50. Itemised ten factors affecting Software Quality and Developer Productivity
51. Differentiate between the Process of Software Validation and Verification
52. State why it is a good idea to test a module in isolation from other modules.